Homework: Advanced Hyperparameter Optimization with Optuna

Objective

In Module 2, we used **Grid Search**. While simple, Grid Search is inefficient: it wastes time checking unpromising combinations and skips the spaces in between your defined grid points.

Your Mission: Implement **Bayesian Optimization** using the **Optuna** library. You will define "richer" search spaces (checking more parameters over continuous ranges) and let Optuna's **TPE (Tree-structured Parzen Estimator)** sampler find the global minimum RMSE for the Ames Housing dataset.

Prerequisites

You will need to install Optuna:

```
conda install optuna -c conda-forge
```

The Task

Create a Python script (or notebook) that performs the following:

1. **Load Data:** Use the same Ames Housing setup (`X_train` , `X_test` , etc.) from Module 2.
2. **Define Objective Functions:** Write an optimization function for **XGBoost**, **LightGBM**, and **CatBoost**.
3. **Optimize:** Run 50-100 trials for each model.
4. **Compare:** Report whether Optuna beat the Grid Search results from the lecture.

1. The Richer Hyperparameter Spaces

Unlike the small grid in class, you must search over these wider, continuous ranges.

A. XGBoost

- `n_estimators` : `Integer(100, 1000)` - How many trees?
- `learning_rate` : `LogUniform(0.001, 0.3)` - Step size (crucial to sample log-uniformly).
- `max_depth` : `Integer(3, 10)` - Complexity of each tree.
- `subsample` : `Uniform(0.5, 1.0)` - Fraction of rows used per tree (prevents overfitting).
- `colsample_bytree` : `Uniform(0.5, 1.0)` - Fraction of columns used per tree.
- `reg_alpha` **(L1)**: `LogUniform(1e-8, 10.0)` - Lasso regularization.
- `reg_lambda` **(L2)**: `LogUniform(1e-8, 10.0)` - Ridge regularization.

B. LightGBM

- `n_estimators` : `Integer(100, 1000)`
- `learning_rate` : `LogUniform(0.001, 0.3)`

- `num_leaves : Integer(20, 150)` - Main complexity parameter for LGBM (replaces max_depth).
- `min_child_samples : Integer(5, 100)` - Minimum data in a leaf to allow a split.
- `subsample : Uniform(0.5, 1.0)`
- `colsample_bytree : Uniform(0.5, 1.0)`

C. CatBoost

- `iterations : Integer(100, 1000)`
- `learning_rate : LogUniform(0.001, 0.3)`
- `depth : Integer(4, 10)`
- `l2_leaf_reg : LogUniform(1e-8, 10.0)` - L2 regularization coefficient.
- `random_strength : Uniform(1e-9, 10.0)` - Randomness for scoring splits (helps escape local minima).
- `bagging_temperature : Uniform(0.0, 1.0)` - Bayesian bootstrap setting.

3. Deliverables

Fill in the table below with your results.

Model	Module 2 Best RMSE (Grid Search)	Homework Best RMSE (Optuna)	Improvement
XGBoost	*[Fill from Mod 2]*	*[Fill]*	*[Calculate]*
LightGBM	*[Fill from Mod 2]*	*[Fill]*	*[Calculate]*
CatBoost	*[Fill from Mod 2]*	*[Fill]*	*[Calculate]*

Reflection Questions

1. **Speed:** Did 50 trials of Optuna feel faster or slower than the Grid Search in Module 2? Why? (Hint: Calculate total fits in Grid Search: `Grid Size * CV Folds`).
2. **Regularization:** Did the best models select high regularization (`reg_alpha` , `reg_lambda`)? If so, what does that imply about the default models (overfitting vs underfitting)?